



BUSINESS REQUIREMENTS & TECHNICAL SPECIFICATION

Google SERP API

Live Google results, AI Overviews and sources as clean JSON — built to be fast, complete and honest about every failure.

Pain points & goals

Solution architecture

System & backend design

Database schema

Tech stack

Delivery plan

VERSION

1.0

DATE

30 September 2026

STATUS

Draft for client review

PREPARED BY

Devfusion Engineering

Document control

Version	Date	Author	What changed
1.0	30 Sep 2026	Devfusion Engineering	First complete draft for client review.

Review and sign-off

Role	Name	Decision	Date
Client product owner			
Client technical lead			
Devfusion engineering lead			

How to read this document

If you are...	Start with
A business stakeholder	Sections 1–4, 15 and 16. They explain the problem, the solution and the plan without technical detail.
An engineer or architect	Sections 5–14. Diagrams first, tables second.
Signing off	Section 17 lists the decisions we need from you.

Related material

- [README.md](#) in the repository: setup, the exact API contract and parsing rules.
- [PROGRESS.md](#): build history, client decisions and what was verified, phase by phase.
- Benchmark reports ([reports/<run_id>/report.md](#)): produced by [bench/run_bench.py](#) on every run.

Contents

1	Summary.....	4
2	Client pain points and how we solve them.....	5
3	Business requirements.....	6
4	Solution architecture (business view).....	8
5	System architecture (technical).....	9
6	Backend architecture.....	11
7	How one request flows.....	13
8	Database design.....	15
9	API specification.....	19
10	Tech stack.....	21
11	Infrastructure and deployment.....	22
12	Non-functional requirements.....	24
13	Security and compliance.....	24
14	Monitoring and alerting.....	25
15	Delivery plan.....	26
16	Risks and mitigations.....	27
17	Open decisions for the client.....	27
18	Glossary.....	28

Figures

Figure 1	Pain points and the fix for each one.....	5
Figure 2	The solution in one picture.....	8
Figure 3	System architecture: edge, API tier, browser worker fleet, data and operations.....	9
Figure 4	Backend on AWS — numbers match the steps in the table that follows.....	11
Figure 5	One request from start to finish, with the time budget for each stage.....	13
Figure 6	Outcome decision tree and error codes.....	14
Figure 7	ER diagram: a search request, its pages and everything read from them.....	16
Figure 8	ER diagram: who sent the request, how it was served, and what it cost.....	17
Figure 9	PostgreSQL tables, keys and relationships.....	18
Figure 10	Tech stack by layer.....	21
Figure 11	AWS deployment across two availability zones, with the delivery pipeline.....	22
Figure 12	Twelve-week plan from smoke test to go-live and hypercare.....	26

A note on numbers

Every figure marked **target** is a goal the benchmark must prove, not a measurement. Live numbers do not exist yet: the development IP is blocked by Google on the first request, so the first real measurement is the proxy smoke test in week 1 (section 15).

1 Summary

The client needs live Google results as clean JSON they can trust: the paid ads, the organic links, the full AI Overview with every source, and the knowledge panel. The engine that does this is **built and tested** against 16 real Google pages. This document describes how to run it as a production service and how we will prove it meets the two numbers that matter.

≥ 98% Valid results per test corpus	≤ 2.0 s P95 response time measured by the client	0 Made-up fields missing = null, incomplete = error	100% Google requests counted in every response
---	--	---	--

All four are targets. They become results after the week-1 smoke test and the week-9 acceptance run.

Where we are today

Built and tested	Waiting on live proxy access	What this document adds
● POST /serp and GET /health with the exact client schema ● 7 parsers checked against 16 real pages ● Completeness gate and honest error codes ● Real destinations for /goto links ● Proxy support by config only ● Benchmark tool and 183 automated tests	● The first run through the client's US residential proxies ● Real valid rate, P95 and traffic per page ● AI Overview wait and expand on live Google ● /goto resolution on Google's own servers	● A production platform on AWS ● A database for keys, usage and results ● Dashboards, alerts and an hourly canary ● A 10-week plan to go-live

The honest summary: the hard part — reading Google's page correctly and refusing to return anything half-done — is solved. What is left is running it at scale through the right proxies and proving the speed target, especially for queries with an AI Overview.

What we need from the client to start

- Proxy credentials and the provider's session syntax for the week-1 smoke test (100 MB budget agreed).
- Answers to the open decisions in section 17.
- Expected daily search volume at launch and in 12 months, to size the platform.

2 Client pain points and how we solve them

These seven problems came up while building and testing the engine. Each one maps to a specific part of the design.



Figure 1 Pain points and the fix for each one

3 Business requirements

3.1 Goals

#	Goal	How we measure it	Target
G1	Reliable data	Valid results ÷ all requests, per corpus	≥ 98%
G2	Fast answers	P95 end-to-end time, failures included	≤ 2,000 ms
G3	Data you can trust	Fields that are guessed or rebuilt	0
G4	Clear usage and cost	Google requests counted in <code>requests_used</code>	100%
G5	Controlled proxy spend	Bytes per page reported; runs stop before the budget	Every run
G6	Room to grow	More capacity by adding workers, not changing code	Linear

3.2 Scope

In scope	Out of scope
- Google web search (<code>google.com/search</code>), US English first - Paid ads (top and bottom blocks) and organic results with sub-links - AI Overview: intro, sections and every source - Knowledge panel, result count, related searches, spelling fixes - More than 10 results through extra pages (<code>&start=10, 20, ...</code>) - JSON, or the raw page HTML when asked - API keys, quotas, usage reports, monitoring, benchmark	- Solving or bypassing CAPTCHAs - Stealth or fingerprint tricks - Third-party SERP APIs, caches or other data sources - Images, News, Shopping and other verticals - Mid-page "in-feed" ads - People Also Ask, local packs, carousels, flight modules - Visiting the result websites themselves

3.3 Business rules

What the platform will never do

- Solve, click or work around a CAPTCHA. A block is reported as `blocked_captcha`.
- Use third-party SERP APIs, scraped caches or any other data source.
- Cache a search result and serve it again. Every answer is a fresh page.
- Make data up. Missing means `null` or empty; incomplete means an error.
- Retry a blocked request on the same IP address.
- Change the client's URL. Even an odd one (like `hl=us`) goes to Google byte for byte.

3.4 Who uses it

User	What they need
Client developers	A stable API, clear error codes, examples and SDKs.
Client SEO / analytics team	Complete, correct data flowing into their tools every day.
Client account owner	Usage and cost per API key, per day, without asking us.
Devfusion operations	Dashboards, alerts and a saved copy of every page that failed.

3.5 Functional requirements

ID	Requirement	Priority
FR-01	Accept POST /serp with url, results, country, language, return_json.	Must
FR-02	Accept only google.com / www.google.com / search links with q=; send them exactly as given.	Must
FR-03	For results above 10, fetch extra pages with &start=10, 20, ... (never num=).	Must
FR-04	Return the agreed shape: status_code, requests_used, elapsed_time, results, error, error_message.	Must
FR-05	Extract paid ads, organic results, AI Overview (intro, sections, sources), knowledge panel, count, suggestions, corrections.	Must
FR-06	Resolve /goto and /url links through Google's own redirect. Never rebuild a link from the breadcrumb.	Must
FR-07	Fail the page (parse_error / aio_incomplete) when anything returned is incomplete.	Must
FR-08	Report blocks, consent walls, degraded pages, timeouts and network failures with the agreed codes.	Must
FR-09	Count every Google request (page loads, retries, /async, /goto) in requests_used.	Must
FR-10	Send debug headers (X-Request-Id, X-Timings, X-Google-Requests, ...) outside the JSON body.	Must
FR-11	Authenticate with X-API-Key. Enforce a rate limit and a monthly quota per key.	Must
FR-12	Keep the page HTML and a screenshot for every request for 14 days.	Should
FR-13	Usage API: requests, valid, failed, Google requests and bytes per key per day.	Should
FR-14	Admin API: create and revoke keys, set quotas, manage proxy providers.	Should
FR-15	Hourly canary with known queries; alert when a field that used to be filled comes back empty.	Should
FR-16	Batch mode: submit many searches, get a webhook when they are done.	Could
FR-17	Client portal for keys, usage and invoices.	Could

3.6 Decisions the client has already confirmed

- Shopping and product cards are page UI, not AI Overview sources.
- Google-hosted sources (e.g. Google Flights) stay. A source that cannot be resolved makes the request incomplete; it is never dropped.
- /goto lookups, retries and AI Overview follow-ups all count in **requests_used**.
- The first live run is a single-session smoke test: it stops at the first CAPTCHA and reports what it did not run.

4 Solution architecture (business view)

This is the picture for non-technical readers: what happens between "send a search link" and "get clean data back".

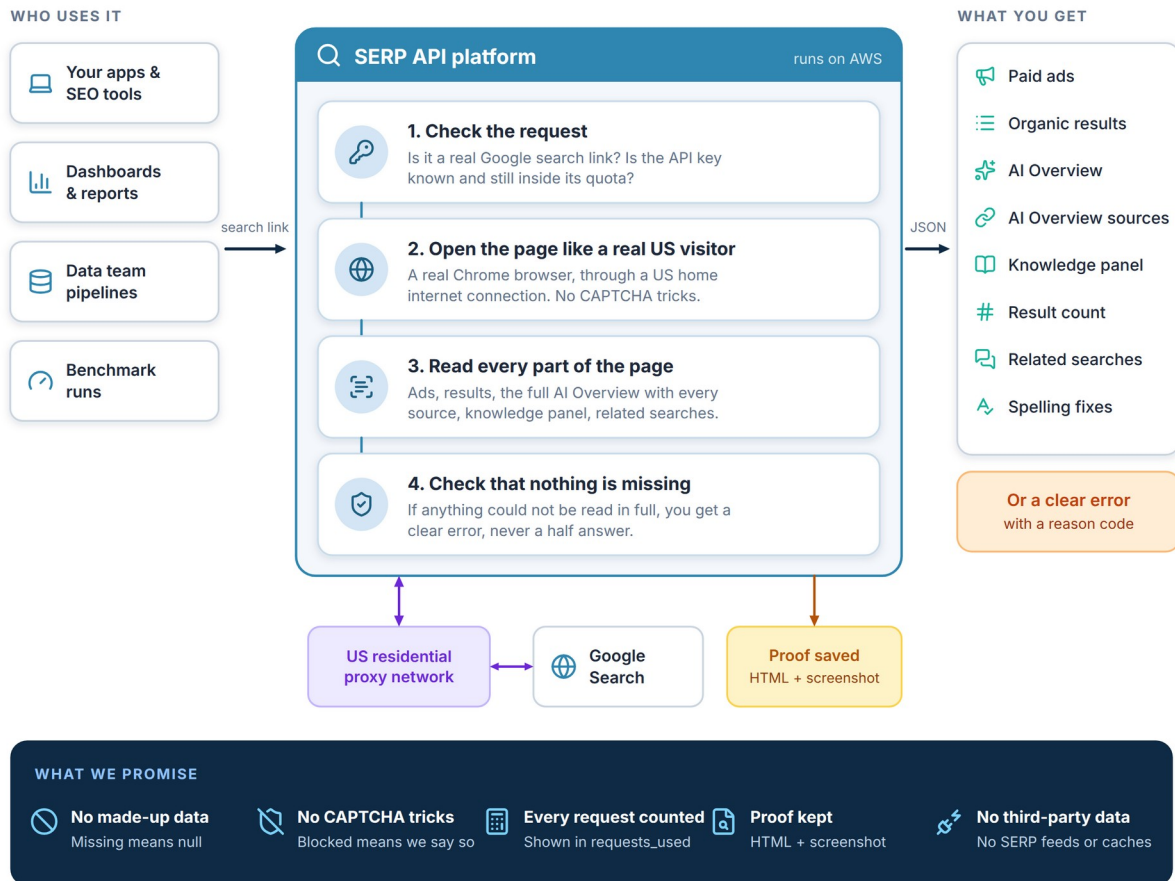


Figure 2 The solution in one picture

In plain words

- You send us a Google search link and your API key.
- We open that exact page in a real browser, through a normal US home internet connection — the way a person would.
- We read everything you asked for and check that nothing is missing or guessed.
- You get clean JSON back, or a clear error that says what went wrong. Either way we keep a copy of the page as proof.

Today (engine)	After go-live (platform)
Runs on one machine	Runs on AWS across two data centres (availability zones)
One browser, one request at a time	Many browser slots in parallel, added and removed automatically
Logs and saved pages on local disk	Dashboards, alerts and a 14-day proof store
One API key in a config file	A key per team, each with its own quota and usage report

5 System architecture (technical)

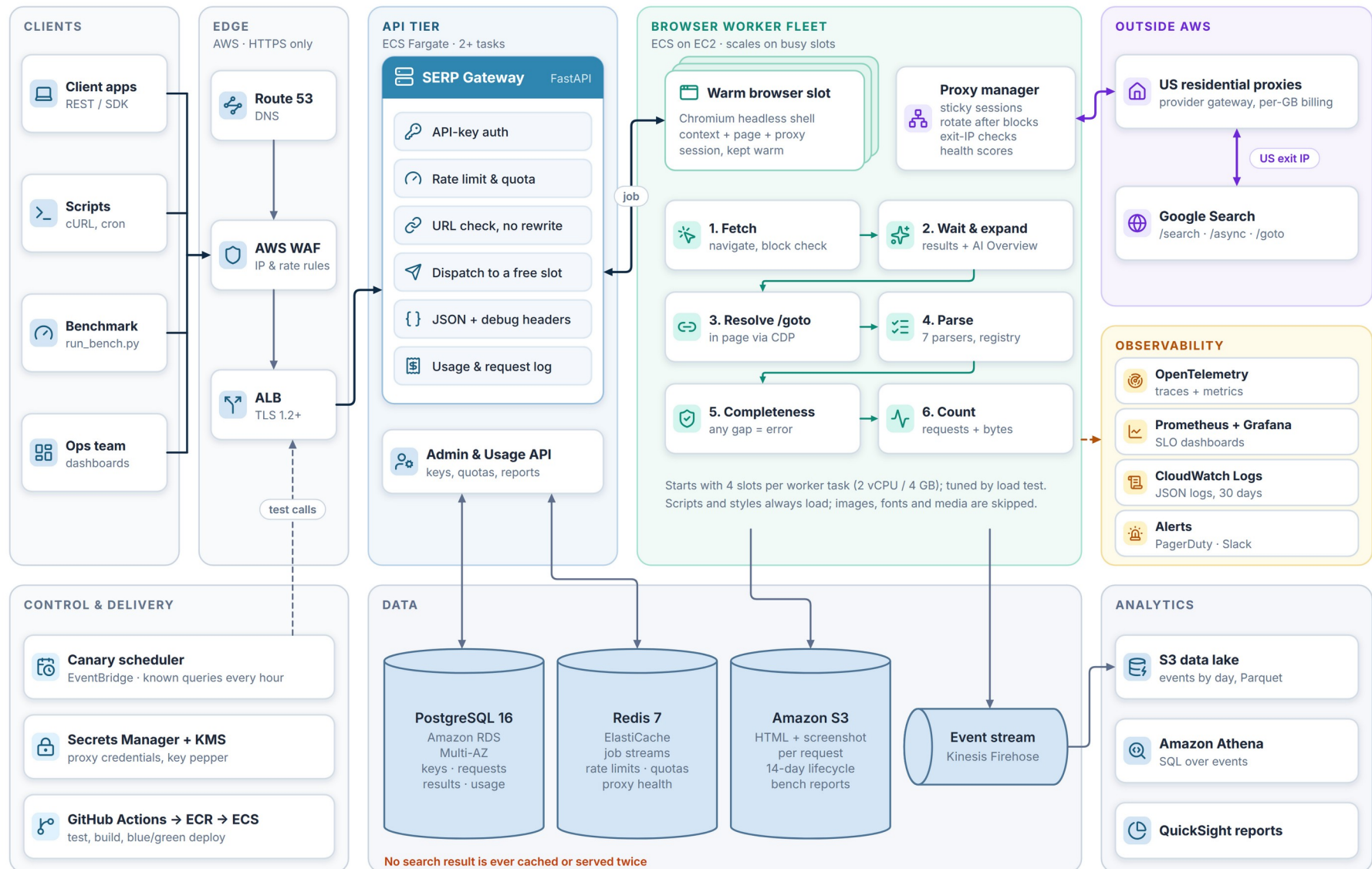


Figure 3 System architecture: edge, API tier, browser worker fleet, data and operations

5.1 Components

Component	What it does	Technology
Edge	DNS, TLS, WAF rules, load balancing	Route 53, ACM, AWS WAF, ALB
SERP Gateway	API keys, quotas, URL check, hands the job to a slot, builds JSON and debug headers	FastAPI on ECS Fargate
Job dispatch	Gives each request to a free warm browser slot and returns the answer	Redis Streams
Browser workers	Load the page, wait, expand the AI Overview, resolve links, parse, verify, count	Playwright + Chromium headless shell on ECS (EC2)
Proxy manager	One sticky session per slot, rotation after blocks, exit-IP checks, health scores	Worker library + Redis
Parser registry	7 parsers: organic, ads, AI Overview, knowledge panel, count, suggestions, corrections	selectolax
Database	Keys, every request, parsed results of valid pages, usage	PostgreSQL 16 (RDS Multi-AZ)
Proof store	Page HTML + screenshot per request	Amazon S3, 14-day lifecycle
Events & analytics	One event per request with timings, bytes and outcome	Kinesis Firehose → S3 → Athena
Observability	Metrics, traces, logs, alerts	OpenTelemetry, Prometheus, Grafana, CloudWatch, PagerDuty

5.2 Why it is built this way

- **API and browsers are split.** An API task is light; a browser slot is heavy. Splitting them lets each scale on its own signal and keeps a browser crash away from the API.
- **One slot = one browser context + one page + one proxy session.** Everything for one page leaves from one IP, the one Google set its cookies on. A session is never swapped in the middle of a request.
- **Headless shell, not full Chrome.** The full browser calls Google on its own (sign-in checks, time sync). That adds traffic and noise the headless shell does not make.
- **Warm browsers.** Starting Chromium takes seconds. Browsers start once and stay open, so a request only pays for the page itself.
- **Redis carries jobs and counters only.** It never stores a search result, so nothing can be served twice by accident.

6 Backend architecture

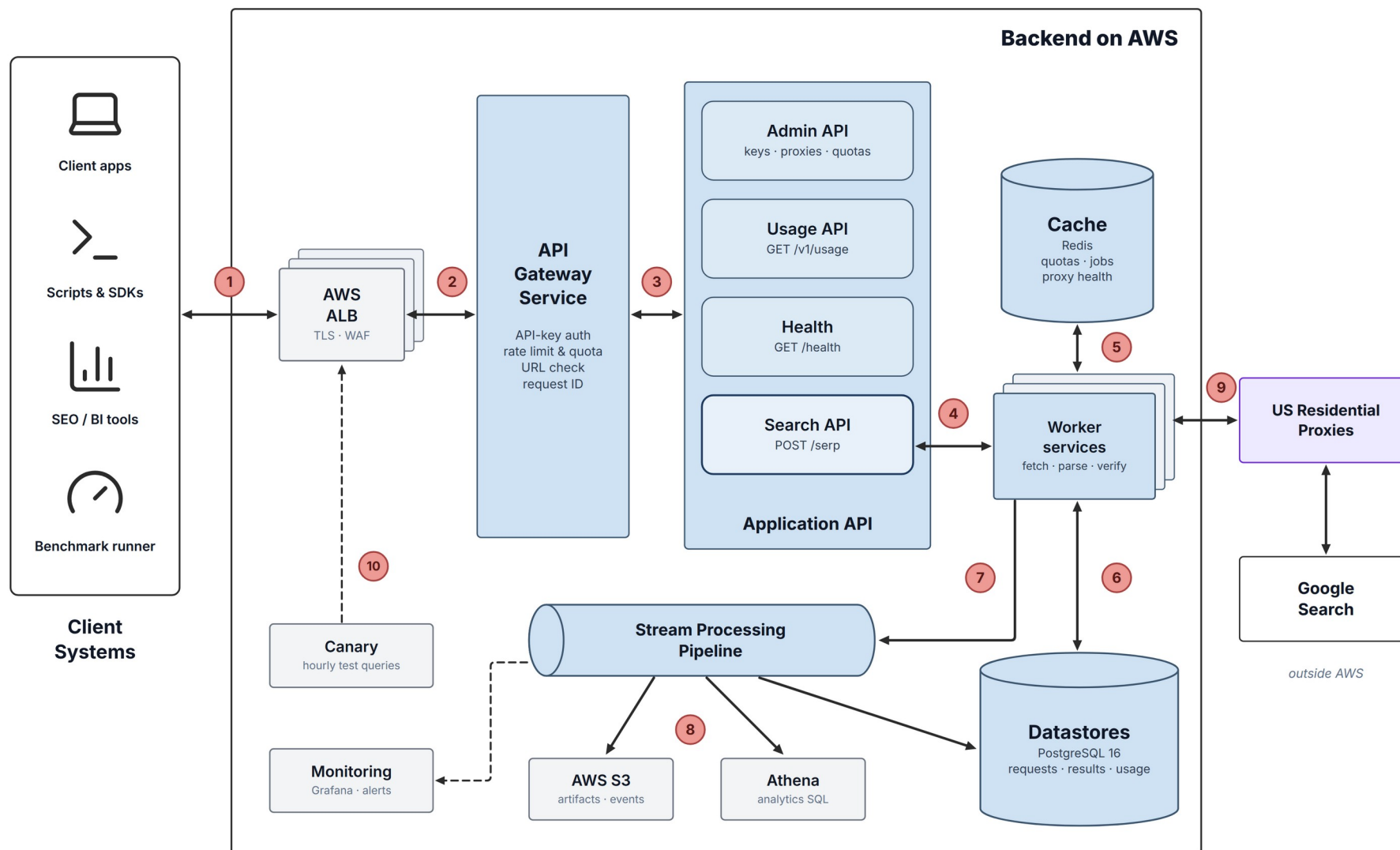


Figure 4 Backend on AWS — numbers match the steps in the table that follows

6.1 What each numbered step does

#	Step	Detail
1	Client call	Client systems call the API over HTTPS with their X-API-Key.
2	Load balancer	The ALB ends TLS, AWS WAF drops abusive traffic, requests are spread across gateway tasks.
3	API gateway	Checks the key, quota and URL, stamps a request ID and routes to the right API.
4	Search API → workers	The job goes to a free warm browser slot through Redis Streams.
5	Cache (Redis)	Holds job streams, rate-limit counters, quotas and proxy health. Never search results.
6	Datastores	Requests, parsed results and usage are written to PostgreSQL.
7	Stream pipeline	Every request emits one event: timings, bytes, Google requests, outcome.
8	S3, Athena, rollups	Events land in S3 as Parquet, Athena queries them, daily usage rolls up into PostgreSQL.
9	Proxies → Google	Workers reach Google only through US residential proxies, one sticky session per slot.
10	Canary & monitoring	Known queries run every hour through the front door; alerts fire on drops in valid rate, speed or parser output.

6.2 Services and their scaling signal

Service	Runs on	Scales on	Start size (prod)
SERP Gateway (Search, Usage, Admin, Health)	ECS Fargate	CPU > 60% or requests per task	2 tasks, 0.5 vCPU / 1 GB
Browser workers	ECS on EC2 (c7i.xlarge)	Busy slots > 70% for 3 min	2 tasks × 4 slots, one per AZ
Event writer / rollups	ECS Fargate (scheduled)	Every 5 minutes	1 task
Canary	EventBridge → ECS task	Hourly	1 task, 5 known queries

Start sizes are a first guess; the week-8 load test sets the real numbers.

7 How one request flows

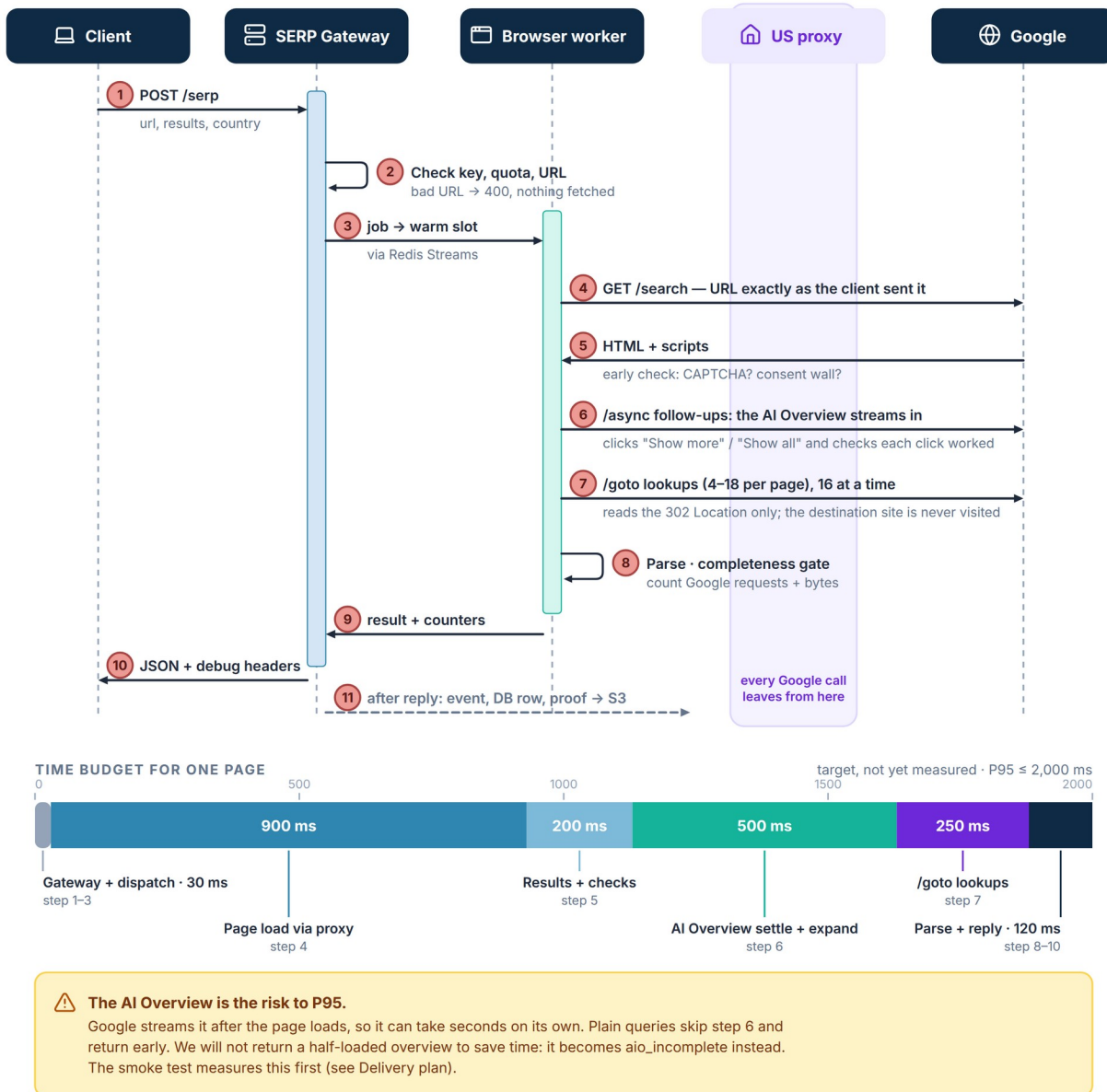


Figure 5 One request from start to finish, with the time budget for each stage

7.1 Every answer is complete or an honest error

The checks run in this order. The first one that fails decides the error code, and nothing partial is ever returned.

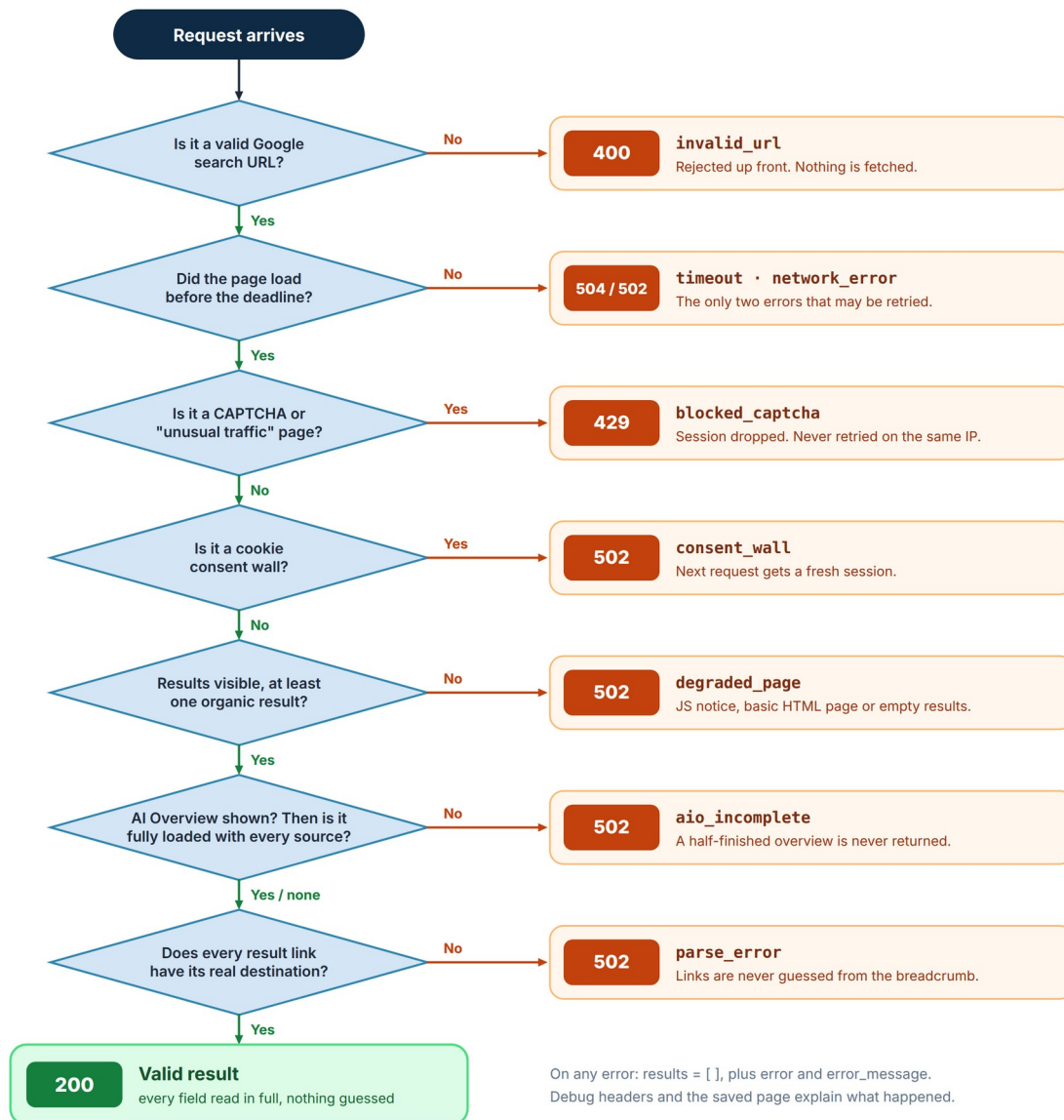


Figure 6 Outcome decision tree and error codes

8 Database design

PostgreSQL holds accounts and keys, every request, the parsed results of valid pages, and usage. Page HTML and screenshots live in S3; the database only keeps their keys. The model is shown three ways: two conceptual ER diagrams (entities, attributes and relationships), then the physical tables.

8.1 Data rules

Rule	Why
Parsed rows are written only for status 200.	A failed page can never look like data. Failures keep the request row and its reason.
<code>aio_sources.url</code> is NOT NULL.	The completeness rule lives in the database too: a source without a real link cannot be stored.
<code>search_requests</code> is partitioned by month.	Fast queries on recent traffic and cheap retention: old partitions are detached, not deleted row by row.
API keys are stored as an 8-character prefix + salted hash.	The key is shown once. A database leak does not leak working keys.
Proxy credentials live only in Secrets Manager.	<code>url_template</code> is stored masked. No secret ever sits in the database or a log line.
<code>google_requests</code> keeps the breakdown as JSONB.	Page loads, <code>/async</code> , <code>/goto</code> , fallbacks, other Google and non-Google calls — <code>requests_used</code> can always be audited.
<code>usage_daily</code> is rolled up from the event stream.	Billing and usage reports never scan raw request rows.

8.2 Retention (proposed)

Data	Where	Kept for
Request rows	PostgreSQL	13 months
Parsed results	PostgreSQL	90 days
Page HTML + screenshots (proof)	Amazon S3	14 days
Request events	Amazon S3 (Parquet)	13 months
Daily usage	PostgreSQL	For the life of the contract
Application logs	CloudWatch Logs	30 days

Retention periods are a proposal. The client confirms them in section 17.

8.3 Conceptual model — search data

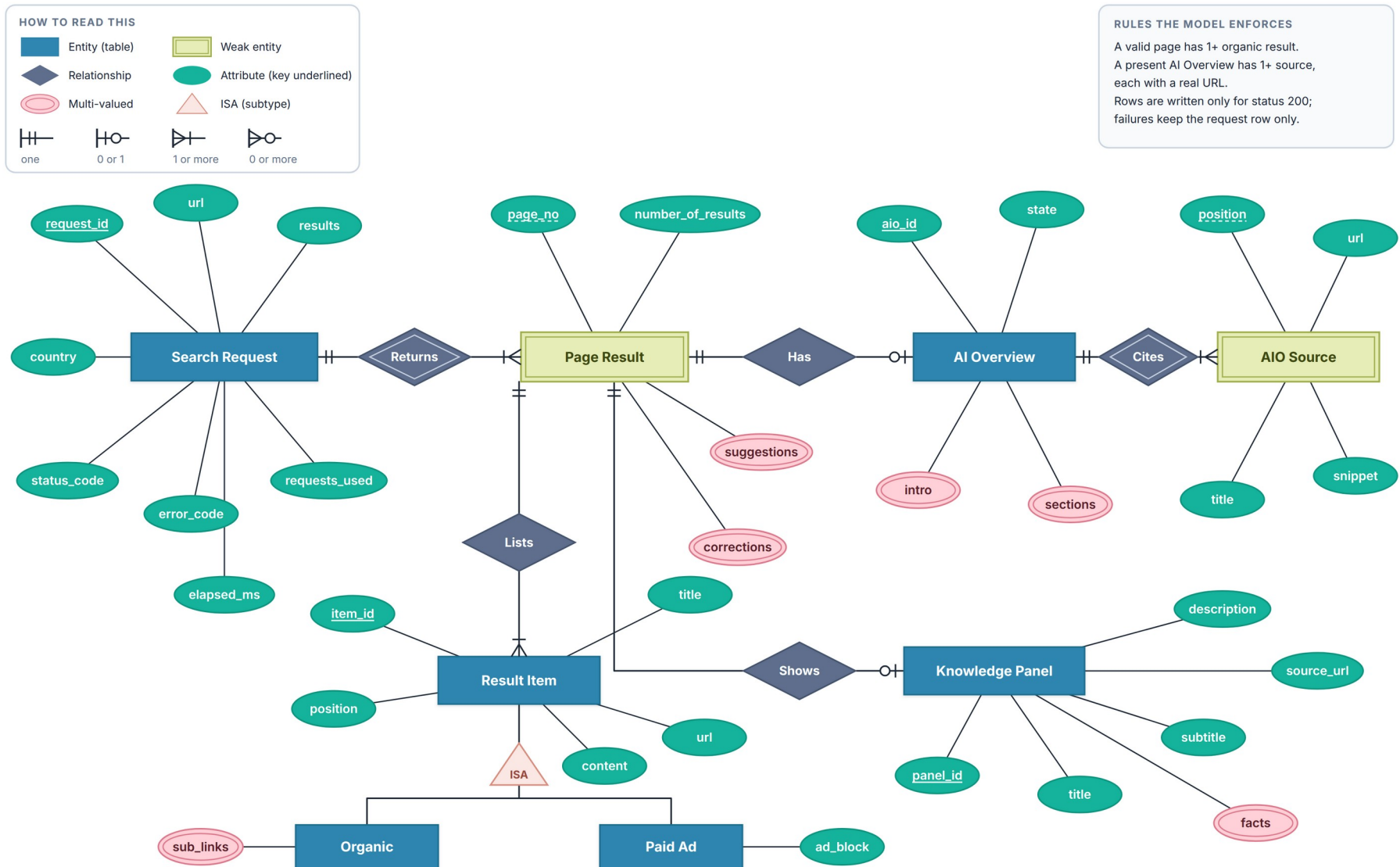


Figure 7 ER diagram: a search request, its pages and everything read from them

8.4 Conceptual model — accounts, proxies and operations

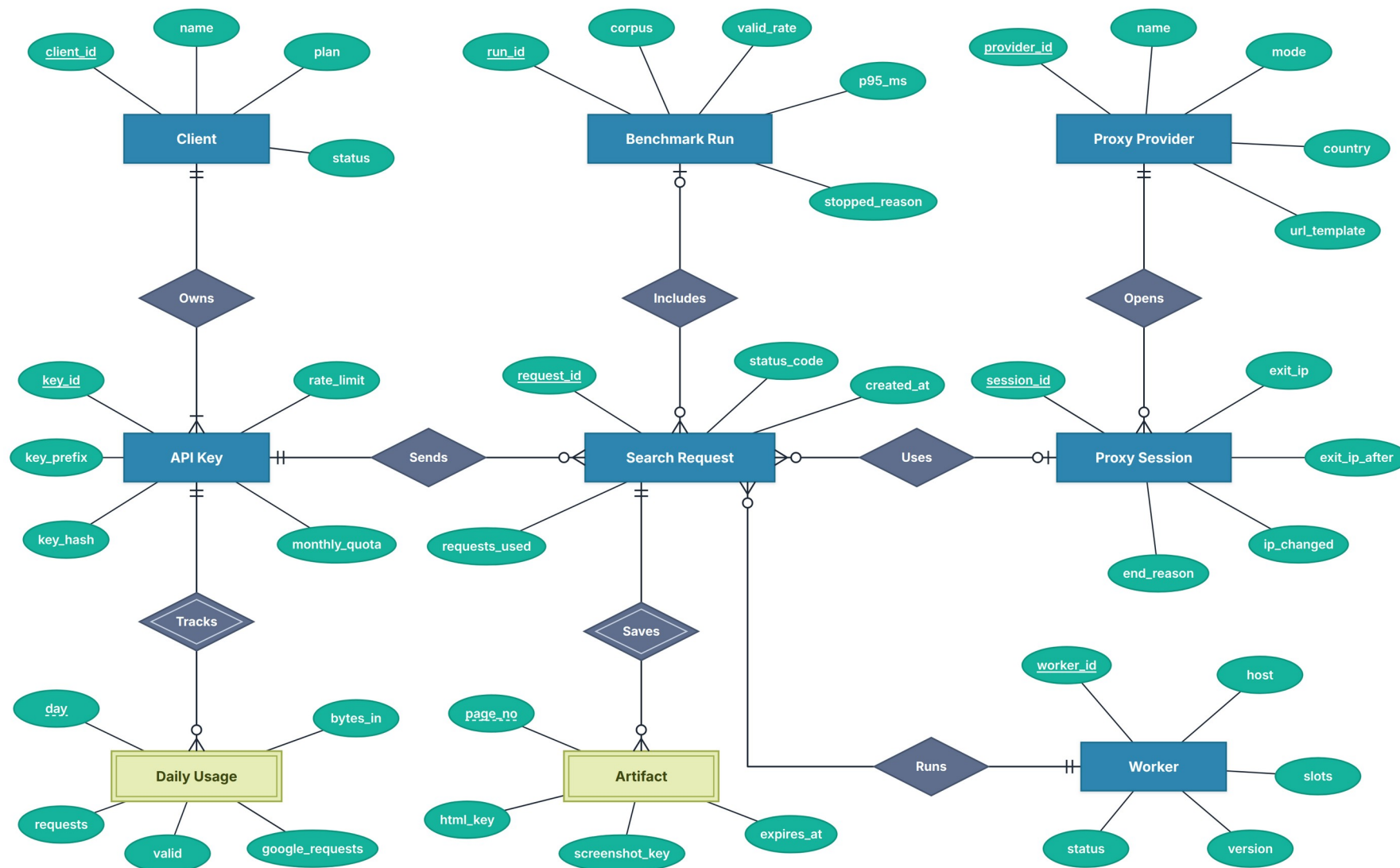


Figure 8 ER diagram: who sent the request, how it was served, and what it cost

8.5 Physical schema

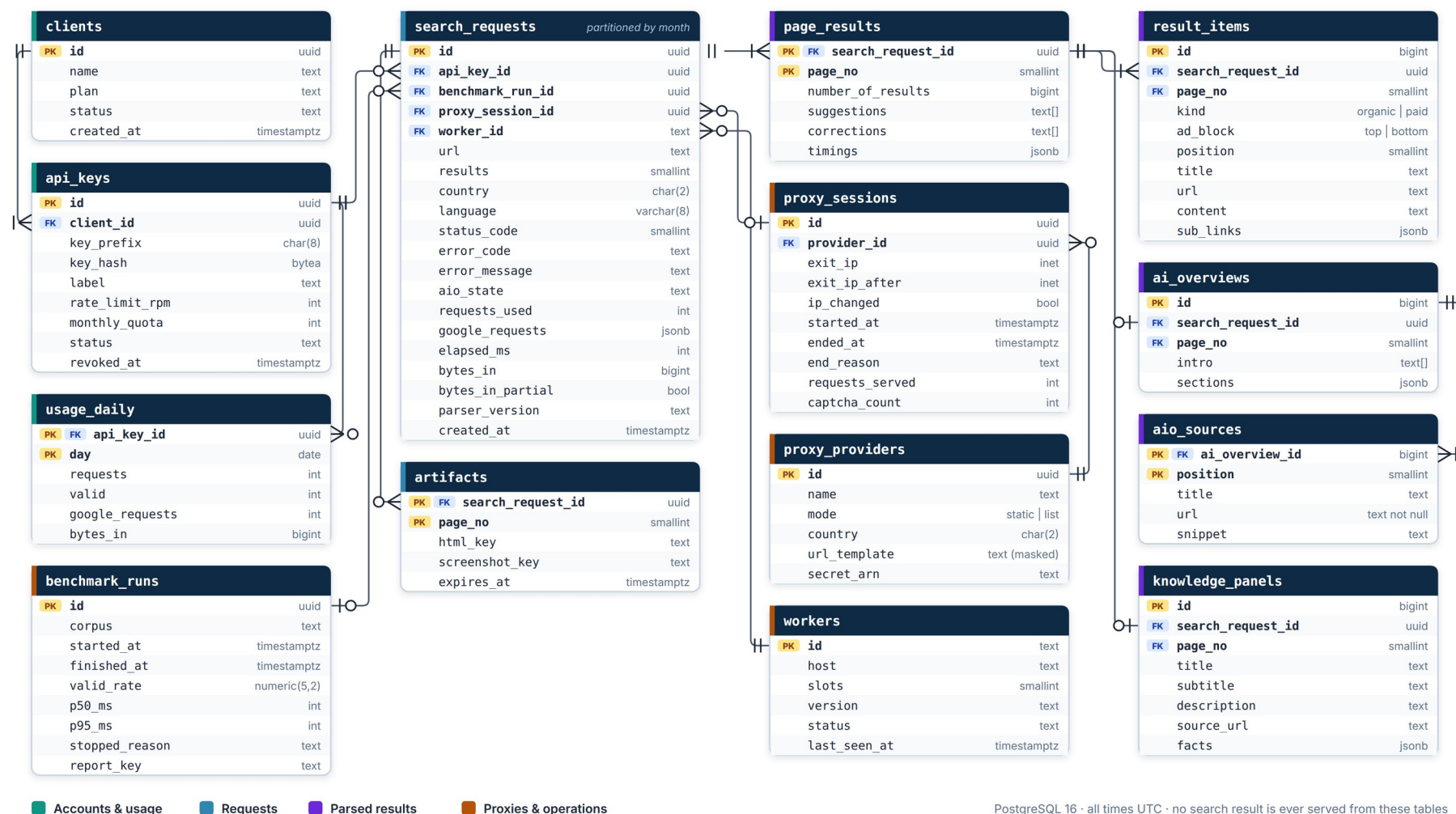


Figure 9 PostgreSQL tables, keys and relationships

9 API specification

9.1 Endpoints

Method	Path	Purpose	Auth	Status
POST	/serp	Fetch and parse one Google results page (or more with results > 10)	X-API-Key	Built
GET	/health	Liveness check for the load balancer	None	Built
GET	/v1/usage?from=&to=	Daily usage for the calling key	X-API-Key	New
POST / DELETE	/v1/keys	Create or revoke keys, set quotas	Admin key	New
POST	/v1/jobs	Batch of searches with a webhook when done	X-API-Key	Phase 2

9.2 Request body — POST /serp

Field	Type	Default	Rule
url	string	—	google.com or www.google.com, path /search, must have q=. Sent exactly as given. Anything else → 400 invalid_url, nothing fetched.
results	int	10	10 = page 1 only. More adds pages with &start=10, 20, ... on a copy of the URL.
country	string	US	Browser locale and proxy country only. Never changes the URL.
language	string	en	Browser Accept-Language only. Never changes the URL.
return_json	bool	true	false returns page 1's rendered HTML as text/html.

9.3 Response — example

```
{
  "status_code": 200, "requests_used": 9, "elapsed_time": 1840,
  "results": [{
    "page": 1,
    "paid": [ { "url": "...", "title": "...", "content": "...", "sub_links": [] } ],
    "organic": [ { "url": "https://...", "title": "...", "content": "...",
      "sub_links": [ { "title": "...", "url": "https://..." } ] } ],
    "ai_overview": {
      "intro": [ "..." ],
      "sections": [ { "title": "...", "text": "- ...\\n- ..." } ],
      "sources": [ { "title": "...", "url": "https://...", "snippet": "..." } ]
    },
    "knowledge_panel": null, "number_of_results": 129000000,
    "suggestions": [ "...", "corrections": []
  ] },
  "error": null, "error_message": null
}
```

Values are illustrative. On any error results is [] and error / error_message explain why.

9.4 Status and error codes

HTTP	error	When
200	null	Valid result: every field read in full.
400	invalid_url	URL not allowed, or a malformed request body. Nothing is fetched.
429	blocked_captcha	Google CAPTCHA or "unusual traffic" page (/sorry/).
502	consent_wall	"Before you continue" consent page.
502	degraded_page	JavaScript notice, basic-HTML page, no results container, or zero organic results.
502	aio_incomplete	AI Overview did not finish loading or expanding, Google shows its "can't generate" message, or a source has no real link.
502	parse_error	Parsing crashed, or a result link's destination could not be resolved.
502	network_error	Browser or network failure.
504	timeout	Navigation or the whole-request deadline ran out.

9.5 Debug headers (never in the JSON body)

Header	Contents
X-Request-Id	ID to quote when asking about a request; also the key of its saved proof.
X-Classification	How the page was judged (ok, blocked, consent, degraded, ...).
X-AIO-State	absent, complete or incomplete.
X-Timings	Stage times (nav_ms, results_ms, aio_ms, links_ms, total_ms), links needed/resolved, bytes in.
X-Google-Requests	requests_used plus every request by kind: document, async, goto, goto_fallback, other_google, non_google.
X-Proxy-Session	Proxy session id, exit IP before and after, whether the IP changed.

10 Tech stack

Green chips are already in the code and covered by tests. Blue chips are added for production.

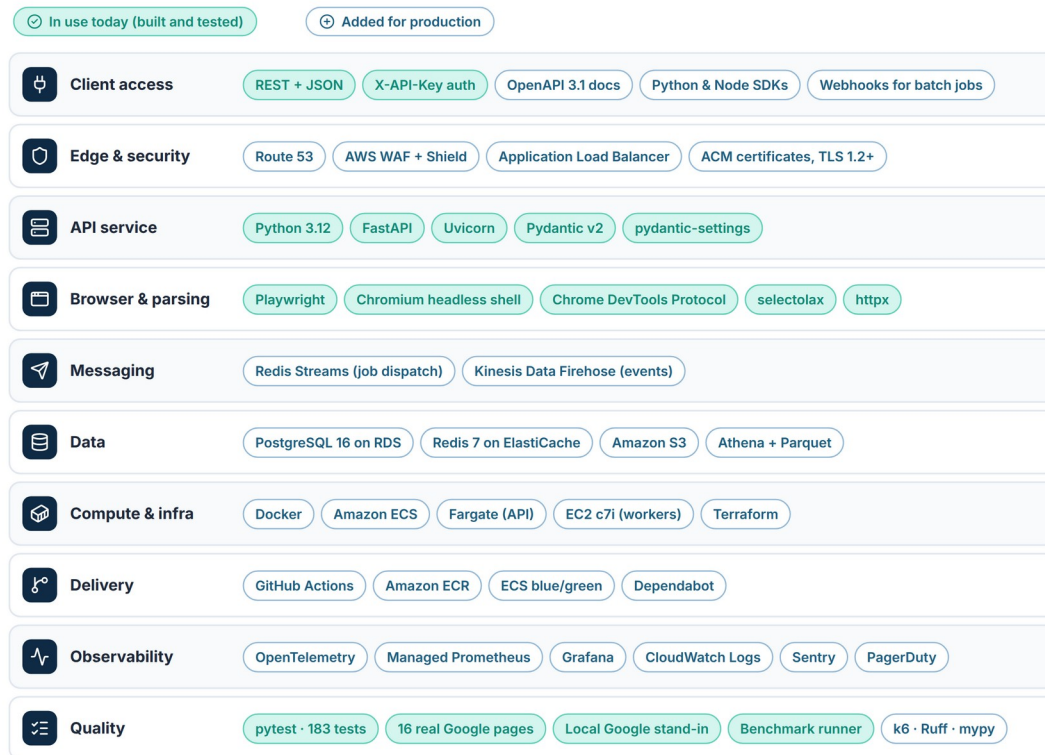


Figure 10 Tech stack by layer

Choice	Why this one
Python 3.12 + FastAPI	The engine is already written and tested in it. Pydantic enforces the client's exact schema.
Playwright + headless shell	A real browser plus DevTools access to count bytes and read redirects. Makes no requests of its own.
selectolax	A few milliseconds per page, so parsing never threatens the time budget.
Redis Streams	Sub-5 ms hand-off to workers, fair work sharing, no extra broker to run.
PostgreSQL 16	Relational core for keys, requests and usage; JSONB for nested parts; partitions for volume.
ECS (Fargate + EC2)	Fargate keeps the API simple; EC2 gives Chromium steady CPU for less. Two services do not need Kubernetes.
Terraform + GitHub Actions	Every environment is rebuilt from code and every change is reviewed in a pull request.
OpenTelemetry + Grafana	Vendor-neutral. Valid rate and P95 are dashboard panels, not log searches.
AWS us-east-1	A US region near Google's US front ends. Confirm against the proxy provider's gateway location.

11 Infrastructure and deployment

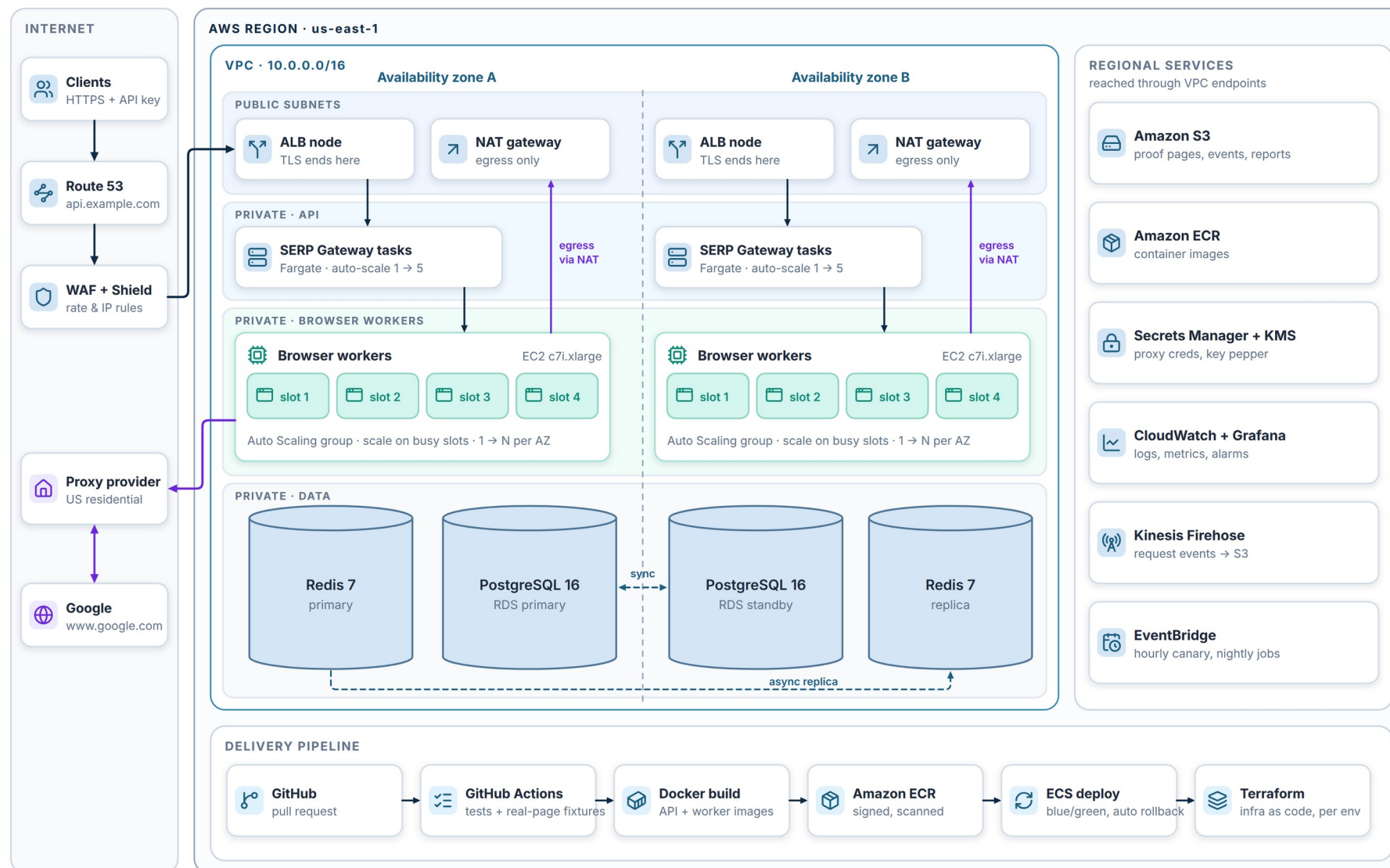


Figure 11 AWS deployment across two availability zones, with the delivery pipeline

11.1 Environments

Environment	Used for	Size	Talks to Google?
Development	Engineers; saved-page fixtures and the local Google stand-in	1 API task, 1 worker (2 slots)	No
Staging	Smoke tests, canary, release checks through the proxy	1 API task, 1 worker (4 slots)	Yes, paced and capped
Production	Client traffic	2+ API tasks, 2+ workers over 2 AZs	Yes

11.2 Capacity planning

Slots needed = peak searches per second × seconds per page × 1.5 headroom. The table assumes 3 seconds per page and a peak of 3× the daily average. Both get replaced by measured values after the smoke test.

Daily searches	Average / s	Peak / s	Busy slots (with headroom)	Worker tasks (4 slots each)
50,000	0.6	1.7	8	2
250,000	2.9	8.7	39	10
1,000,000	11.6	34.7	156	39

Proxy traffic per page is not known yet; it is the first thing the smoke test measures. Proxy cost = pages × measured MB per page × provider price.

11.3 Release flow

- Every pull request runs the 183 tests, including the 16 real Google pages and the local Google stand-in. Nothing reaches staging if a parser test fails.
- Images are built once, scanned and pushed to ECR, then promoted unchanged from staging to production.
- ECS deploys blue/green. If the error rate or health checks slip, it rolls back on its own.
- A parser fix (Google changed its page) is a normal release: saved failing page → new fixture → fix → same-day deploy.

12 Non-functional requirements

Area	Requirement	Target
Performance	End-to-end P95 per corpus, measured by the client, failures included	≤ 2,000 ms
Reliability	Valid-result rate per corpus	≥ 98%
Completeness	Returned items with every field read in full	100% (else error)
Availability	API reachable and answering (gateway + health)	99.9% monthly
Scalability	Capacity grows by adding worker slots	Config only
Transparency	Every Google request counted and broken down by kind	100%
Cost control	Proxy bytes per page measured and reported; test runs stop before budget	Every request
Traceability	Every failure has a request ID, a reason and a saved page	100%
Recovery	Data loss window / time to restore	RPO ≤ 5 min, RTO ≤ 1 h
Maintainability	Google layout change → tested fix in production	Same working day

13 Security and compliance

Area	What we do
Transport	TLS 1.2+ only, HSTS on the API domain.
Authentication	API keys: 8-character prefix + salted hash; shown once. Per-key rate limit and quota. Admin API needs a separate key and an IP allow-list.
Secrets	Proxy credentials in AWS Secrets Manager (KMS-encrypted). Masked in every log line (http://use***@host:port).
Network	Only the load balancer is public. Workers and data sit in private subnets; workers leave through NAT to the proxy provider only.
Data	No personal data is collected. Search terms in URLs are treated as client-confidential. Encryption at rest for RDS, Redis and S3.
Access	Least-privilege IAM roles per service, SSO for engineers, CloudTrail audit log.
Responsible use	No CAPTCHA solving, no stealth or fingerprint spoofing, pacing and traffic budgets in every test tool.

For the client's legal review

Google's terms of service restrict automated access to its search results. The platform avoids evasion techniques on purpose, but the decision to collect this data, and at what volume, sits with the client and their legal advisers.

14 Monitoring and alerting

Two numbers run the service: the valid-result rate and P95. Everything else explains why one of them moved.

Signal	Alert when	First action
Valid-result rate (15 min)	Below 98%	Look at the top failure reason and its saved pages.
P95 latency (15 min)	Above 2,000 ms	Check stage timings (X-Timings) to see which stage grew.
blocked_captcha share	Above 2%, or a spike on one exit IP	Rest that proxy or provider; confirm sessions rotate.
aio_incomplete share	Doubles against the 7-day baseline	Check AI Overview wait settings against the canary pages.
Canary field drift	A field that used to be filled comes back empty	Parser fix from the saved page, same day.
Proxy bytes per page	30% above baseline	Check what the page started loading; tighten blocking.
Busy worker slots	Above 80% for 10 minutes	Autoscaling should already be adding workers; page on-call if not.
Gateway 5xx (own errors)	Above 0.5%	Roll back the last release if it lines up.

Thresholds are starting points and will be tuned on real traffic.

15 Delivery plan

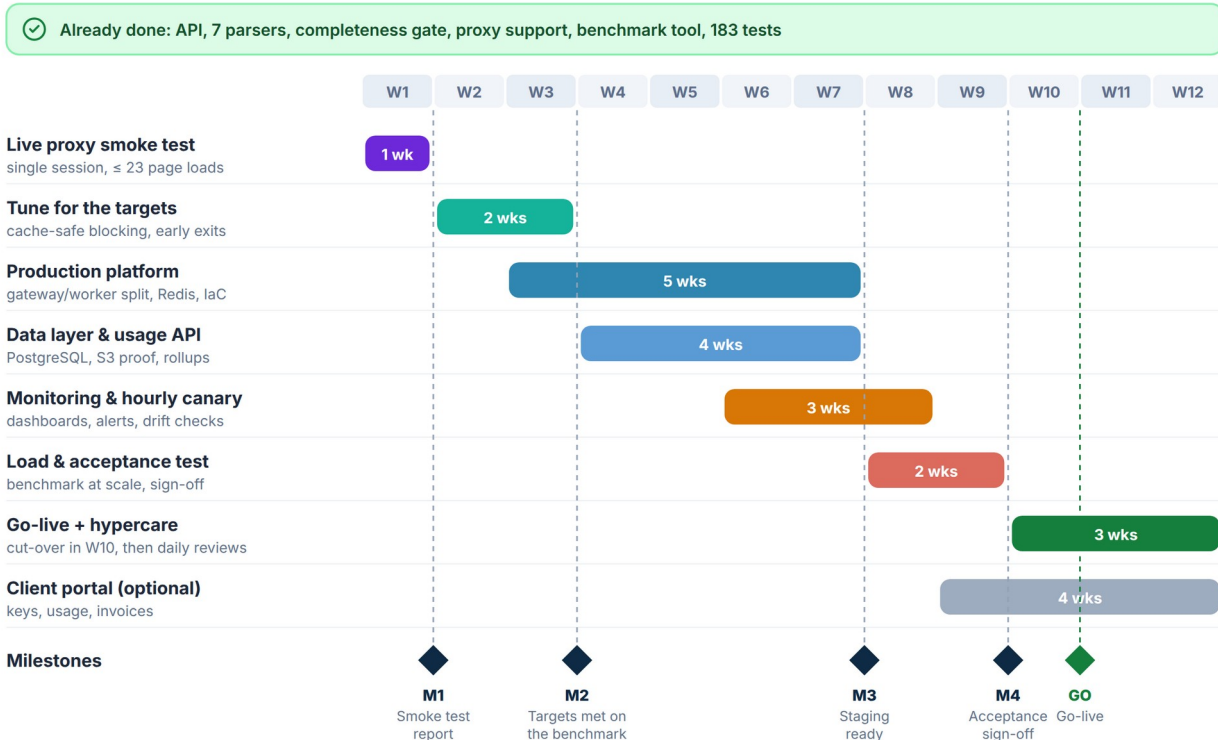


Figure 12 Twelve-week plan from smoke test to go-live and hypercare

Milestone	When	Done when
M1 Smoke test report	End of W1	Up to 23 page loads through the client's proxy. The report shows traffic per page, block rate, AI Overview timing, and every query that did not run.
M2 Targets met on the benchmark	End of W3	Mixed and difficult corpora at ≥ 98% valid and P95 ≤ 2 s — or a written gap analysis with options and costs.
M3 Staging ready	End of W7	The full platform built from Terraform in staging; canary green for 3 days in a row.
M4 Acceptance sign-off	End of W9	Load test at the agreed volume passes; the client signs acceptance.
Go-live	End of W10	Production traffic on, followed by two weeks of daily failure reviews.

16 Risks and mitigations

Risk	Impact	Likelihood	Mitigation
Google blocks proxy traffic at scale	High	Medium	Health scores, fresh sessions after a block, block rate per exit IP, a second provider on standby.
AI Overview pushes P95 over 2 s	High	High	Early "no AI Overview coming" exit, tuned waits. If still over, agree a separate P95 for AI Overview queries — completeness is not traded for speed.
Google changes the page layout	High	Medium	Hourly canary, 16-page fixture suite, same-day fix from the saved page.
/goto lookups change or get limited	High	Medium	In-page lookup with a fallback client; alert on unresolved links; never guess a URL.
Proxy bandwidth costs more than planned	Medium	Medium	Bytes measured per page, cache-safe resource blocking, hard budgets on test runs.
Proxy provider outage	Medium	Low	Provider switch is config only (static / list); keep a second contract ready.
Terms-of-service exposure	High	Client to assess	No evasion techniques; pacing; client legal review before scaling up.

17 Open decisions for the client

We need an answer on these before acceptance. Each has our recommendation so a "yes" is enough.

#	Question	Our recommendation
1	A result link whose destination cannot be resolved: keep <code>parse_error</code> , or add a new <code>incomplete_result</code> code?	Keep <code>parse_error</code> now; add the new code in v1.1 if your tooling needs to tell them apart.
2	With <code>results > 10</code> , page 1 succeeds but page 2 is blocked: return an error for all, or the good pages plus a flag?	Return the error (current behaviour), matching "results: [] on error".
3	Google shows "Can't generate an AI Overview right now": failure or no AI Overview?	Failure (<code>aio_incomplete</code>), because the page is not what a user normally sees.
4	Bold-labelled groups inside AI Overview lists: separate sections or lines inside their section?	Lines inside their section (- <code>Label: text</code>), as built.
5	Malformed request body: <code>400 invalid_url</code> ?	Yes, it is the closest agreed code.
6	Mid-page "in-feed" ads: include in paid?	No, paid stays top and bottom blocks as the brief defines.
7	A separate P95 target for queries with an AI Overview?	Decide after the smoke test shows real AI Overview timings.
8	Retention periods in section 8.2.	Accept as proposed.
9	Expected daily volume at launch and in 12 months.	Needed to size workers and proxy spend (section 11.2).

18 Glossary

Term	Meaning
SERP	Search Engine Results Page — the page Google shows after a search.
AI Overview	The AI-written summary Google shows above the results, with links to its sources.
Organic result	A normal, unpaid search result.
Knowledge panel	The fact box on the right side for people, places and things.
CAPTCHA	Google's "prove you are human" check. We never try to solve it.
Residential proxy	A connection that leaves from a normal home internet address instead of a data centre.
Sticky session	Keeping the same proxy exit IP for all requests that belong to one page.
/goto link	Google's wrapper around result links since 26 Aug 2026. We ask Google where it points instead of guessing.
P95	95% of requests are faster than this number.
Valid result	A 200 response where every returned field was read in full.
Canary	A small set of known searches run on a schedule to catch problems before clients do.
Fixture	A saved real Google page used in automated tests.
Headless shell	Chromium without a window, used for automation. Unlike the full browser, it makes no calls to Google on its own.
CDP	Chrome DevTools Protocol — how the worker counts bytes and reads redirects inside the browser.